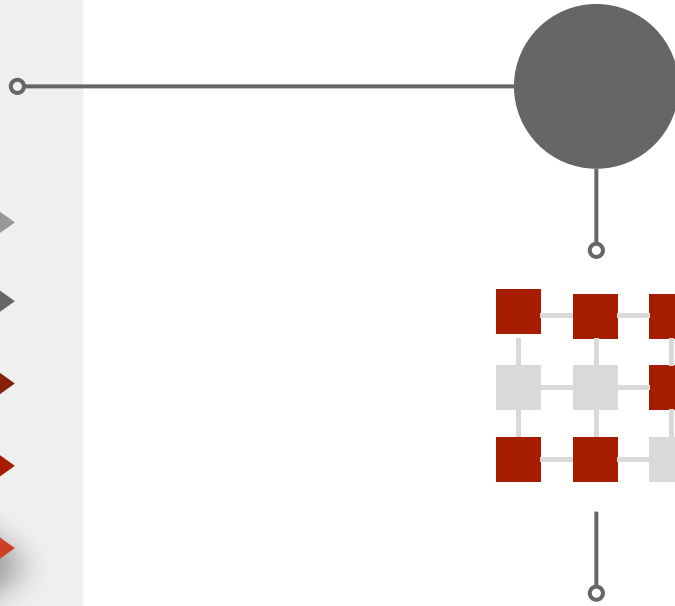
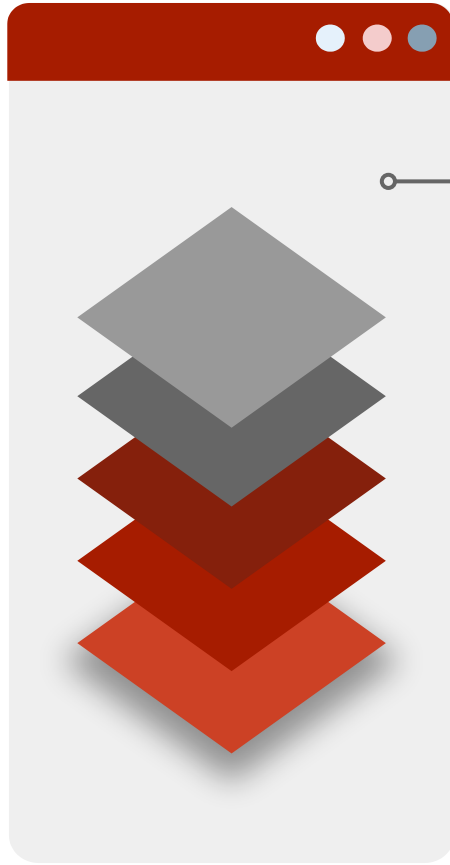




Session 8

Joining with a one-to-many relationship

Joining parts and inventories

For now, we know that *parts* is a list of all LEGO parts, and a new table, *inventory_parts*, has some additional information about those parts, such as the *color_id* of each part you would find in a specific LEGO kit.

Let's join these two tables together to observe how joining *parts* with *inventory_parts* increases the size of your table because of the one-to-many relationship that exists between these two tables.

Joining in either direction

An `inner_join` works the same way with either table in either position. The table that is specified first is arbitrary, since you will end up with the same information in the resulting table either way.

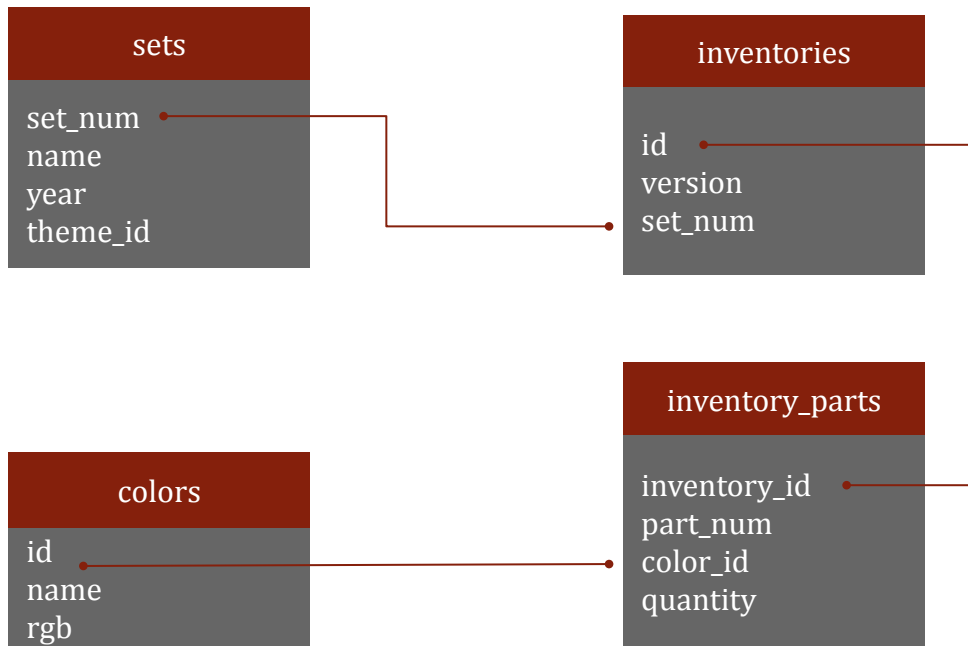
Let's prove this by joining the same two tables from the last exercise in the opposite order!

Joining three or more tables

You can string together multiple joins with `inner_join` and the pipe (`%>%`), both with which you are already very familiar!

We'll now connect *sets*, a table that tells us about each LEGO kit, with *inventories*, a table that tells us the specific version of a given set, and finally to *inventory_parts*, a table which tells us how many of each part is available in each LEGO kit.

Joining three or more tables



Joining three tables

- Combine the *inventories* table with the *sets* table.
 - Next, join the *inventory_parts* table to the table you created in the previous join by the *inventory* IDs.
-

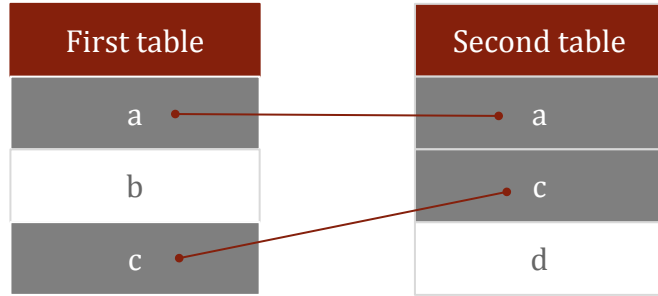
What's the most common color?

Now let's join an additional table, *colors*, which will tell us the color of each part in each set, so that we can answer the question, "what is the most common color of a LEGO piece?"

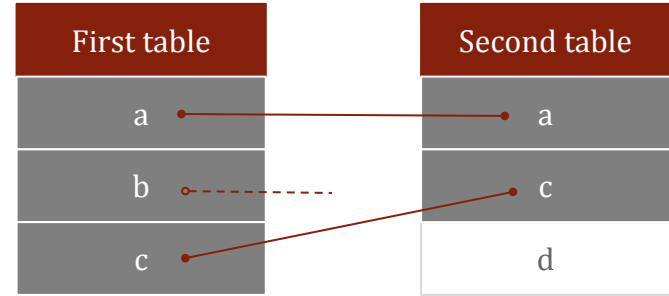
- Inner join the *colors* table using the *color_id* column from the previous join and the *id* column from *colors*; use the suffixes "_set" and "_color".
- Count the *name_color* column and sort the results so the most prominent colors appear first.

Left and Right Joins

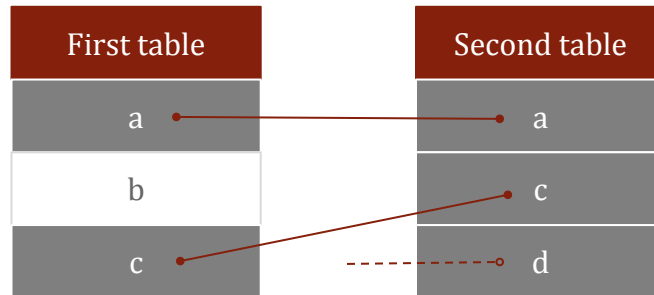
Inner Join



Left Join



Right Join





Break
15 minutes

Left joining two sets by part and color

Now you'll practice your ability to do this looking at two new sets: the Millennium Falcon and Star Destroyer sets.

```
millennium_falcon <- inventory_parts_joined %>%  
  filter(set_num == "7965-1")
```

```
star_destroyer <- inventory_parts_joined %>%  
  filter(set_num == "75190-1")
```

Left joining two sets by color

The last exercise, you joined two sets based on their part and color. What if you joined the datasets by color alone?

- Sum the *quantity* column by *color_id* in the Millennium Falcon dataset.
- Now, sum the *quantity* column by *color_id* in the Star Destroyer dataset.
- Left join the two datasets, *millennium_falcon_colors* and *star_destroyer_colors*, using the *color_id* column and the *_falcon* and *_star_destroyer* suffixes.

The right-join verb

Counting part colors

Sometimes you'll want to do some processing before you do a join, and prioritize keeping the second (right) table's rows instead. In this case, a right join is for you.

In the example below, we'll count the *part_cat_id* from *parts*, before using a *right_join* to join with *part_categories*. The reason we do this is because we don't only want to know the count of *part_cat_id* in *parts*, but we also want to know if there are any *part_cat_ids* not present in *parts*.

- Use the count verb to count each *part_cat_id* in the *parts* table.
- Use a *right_join* to join *part_categories*. You'll need to use the *part_cat_id* from the count and the *id* column from *part_categories*.
- Use *replace_na* to replace NAs in the *n* column with the value 0.